Enhancing Code Retrieval Accuracy in Developer-Oriented RAG Systems using Semantic Chunking

Name of Authors:

Rudraksh Kapoor
Chitkara University of Institute of Engineering and Technology
Chitkara University, Punjab, India
rudraksh746.be22@chitkara.edu.in

Anshul Arora
Chitkara University of Institute of Engineering and Technology
Chitkara University, Punjab, India
anshul141.be22@chitkara.edu.in

Abhishek
Chitkara University of Institute of Engineering and Technology
Chitkara University, Punjab, India
abhishek43.be22@chitkara.edu.in

Abstract— Retrieval-Augmented Generation (RAG) is a technique that augments large language models (LLMs) with access to external knowledge stores, enabling domain-specific responses without retraining. Current RAG pipelines commonly apply fixed-size character chunking methods that are agnostic to content semantics, causing severe context fragmentation when applied to structured source code. This paper presents a controlled experiment comparing two retrieval pipelines over the Flask Python library codebase: Pipeline A (baseline) uses a fixed-size recursive character chunking strategy, while Pipeline B (proposed) uses an Abstract Syntax Tree (AST)-aware chunking method that aligns chunk boundaries with Python syntactic units. Both pipelines employ ChromaDB as the vector store and the all-MiniLM-L6-v2 model for document embeddings. Evaluated across 20 developer-focused queries, the AST-based approach consistently retrieved semantically complete code units—including full class definitions, function signatures, and docstrings—whereas the baseline produced fragmented, mid-function snippets in four documented failure cases. The proposed approach resolves two critical RAG failure modes: context fragmentation and keyword blindness. Results confirm that AST-aware chunking is a necessary preprocessing step for high-quality code-based RAG systems.

Keywords—Retrieval-Augmented Generation (RAG), Semantic Chunking, Code Retrieval, Vector Databases, ChromaDB, LangChain, Abstract Syntax Tree (AST), Flask, Large Language Models (LLMs)

I. Introduction

The rise of large language models (LLMs) used for code generation and to assist developers has transformed how software engineers view their productivity when coding. Companies such as GitHub (GitHub Copilot), OpenAI (ChatGPT), and DeepSeek (DeepSeek-Coder) are able to use LLMs to generate syntactically correct code from natural language queries. However, LLMs rely solely on encoded parametric information, which means that they do not have the ability to adapt to proprietary, private, or rapidly evolving codebases that they were not trained with at the time of development [1].

Retrieval-Augmented Generation (RAG) attempts to improve this limitation by combining the generative aspects of LLMs with an additional retrieval step that pulls related documents from an outside knowledge store at inference time [2]. RAG has been very effective in many textual domains, including question answering, verifying the accuracy of information, and summarizing documents. However, applying RAG to the domain of source code retrieval in a structured hierarchical manner has not been significantly researched or performed exceptionally because of a fundamental mismatch between LLMs chunking the source code based on general language principles vs. the syntactical rules established by programming languages [3].

The problem at hand is how to classify code into discrete retrievable pieces according to a formalised coding convention. Chunking code using the default method in RAG pipelines divides code's characters into predetermined units irrespective of code's syntactic structure. Consequently, function definitions will often be split, as will class declarations and their associated method bodies, and docstrings will be cut from their function signatures. The fragments that result from this method will be syntactically incomplete and semantically misleading, rendering them almost worthless in providing any context necessary for a developer to pose questions to an LLM [4].

In this paper, we consider whether substituting the default character-based chunking method with an AST-accurate semantic splitting process will improve code retrieval from a developer-focused RAG system. As part of this study, we build two parallel retrieval pipelines over the entire Flask web framework Python source code, presently consisting of 83 .py files. Pipeline A serves as the control by using LangChain's RecursiveCharacterTextSplitter to create chunks of 1,000 characters each with a 200-character overlap. Pipeline B employs LangChain's language-aware splitter, which gives top priority to finding breaks in chunks based on class and function definitions, with any chunk size

restriction applied only when no class or function definition break exists. Both pipelines generate embeddings using the all-MiniLM-L6-v2 sentence transformer model stored in a ChromaDB vector database.

This paper provides the following primary contributions: (i) a comparative assessment of fixed-size and AST chunking techniques applied to the retrieval of source code; (ii) an enumeration and characterization of four primary failure modes existing within baseline retrieval systems; (iii) empirical evidence demonstrating that semantic-based chunking creates better-quality retrieval units of entirely logical code; and (iv) an experimental framework for running tests using publicly available libraries (LangChain, ChromaDB, and HuggingFace) on a real-world Python codebase.

The rest of the paper is as follows: Section II provides a review of previous research, Section III presents details about the proposed methodology, Section IV outlines the experimental setup, Section V discusses the results, and Section VI presents conclusions and future work directions.

II. Literature Review

This section provides a review of foundational and recent research on Retrieval-Augmented Generation (RAG), code generation with Large Language Models (LLMs), and chunking strategies for retrieving technical documents.

A. Retrieval-Augmented Generation (RAG)

Lewis et al. [2] are credited with creating the first RAG framework, which fused a dense passage retrieval system with a sequence-to-sequence generator. Their research showed a large improvement in knowledge-intensive Natural Language Processing (NLP) tasks. Izacard et al. [5] explored how varying amounts of retrieved passages affect generation quality and laid the groundwork for multi-document RAG applications. Asai et al. [6] introduced Self-RAG, enabling the LLM to decide when to retrieve and whether retrieved passages are useful, significantly reducing irrelevant context in output.

B. Code Generation with LLMs

Chen et al. [7] introduced Codex and the HumanEval benchmark, establishing a structural and evaluative framework for LLM-based code generation. Li et al. [8] demonstrated that large-scale training with AlphaCode could solve competitive-level programming problems. More recent code-specific model architectures such as DeepSeek-Coder [9] and StarCoder2 [10] use the HumanEval and MBPP benchmarks for evaluation; however, all such systems are limited by their reliance on parametric knowledge alone and require RAG integration for production developer tooling.

C. Code-Specific RAG Systems

CODERAG-BENCH, the most extensive benchmark of retrieval-augmented code-generation tasks, was created by Wang et al. [3] and covers fundamental, general, and repository-level programming tasks. Evaluating 10 retrieval and 10 generation models, the authors concluded that while high-quality retrieval improves code generation quality, most retrieval models today do not provide sufficiently useful information to support code generation. They also found that chunking strategy is one of the most important variables, with chunk sizes between 200 and 800 tokens typically producing the greatest success. Zhou et al. [11] introduced DocPrompting, which uses retrieved library documentation to support code generation, reporting up to a 6.9% improvement in code generation for harder library subsets of ODEX.

D. Chunking Strategies for RAG

Standard text chunking algorithms such as RecursiveCharacterTextSplitter operate on character count alone without regard for content structure. They are suited for prose but do not align well with structured code where logical syntactic units—functions, classes, and methods—should ideally remain intact [4]. Ding et al. [14] showed that function-level granularity outperforms both file-level and line-level retrieval for cross-file code completion. Su et al. [15] developed active retrieval based on generation context to dynamically select code chunks, though they do not compare AST-aware boundary detection with character-based chunking in a controlled setting—a gap that this paper directly addresses.

E. Comparison of Related Works

The differences between different prior code retrieval methods are summarized in Table I below. Each entry shows the respective chunking technique, the respective code library being retrieved, the vector store being used, and any remaining limitations in the previous work and how the current proposed work intends to fix those issues. This work is unique in that it is the only prior paper that has simultaneously used AST-aware semantic chunking on a Python code library corpus, and stored it using ChromaDB. Furthermore, to our knowledge, no prior studies have utilized these two code retrieval methods, so the current paper's aim is to fill this gap by using both methods under a controlled experimental test.

TABLE I. Comparison of Related Works on Code Retrieval in RAG Systems

Reference	Chunking Method	Domain	Vector Store	Limitation
Lewis et al. [2]	Fixed-size passage	General NLP	FAISS	No code support
Wang et al. [3]	Token-window (200–800)	Code (multi-task)	Multiple	No AST awareness
Zhou et al. [11]	Document-level	Library docs	BM25/Dense	No function isolation
Zhang et al. [12]	Function-level	Repository code	BM25	Iterative only
Ding et al. [14]	Function-level	Cross-file code	Embedding	No comparison study
Proposed Work	**AST-aware semantic**	**Python Library**	**ChromaDB**	**None identified**

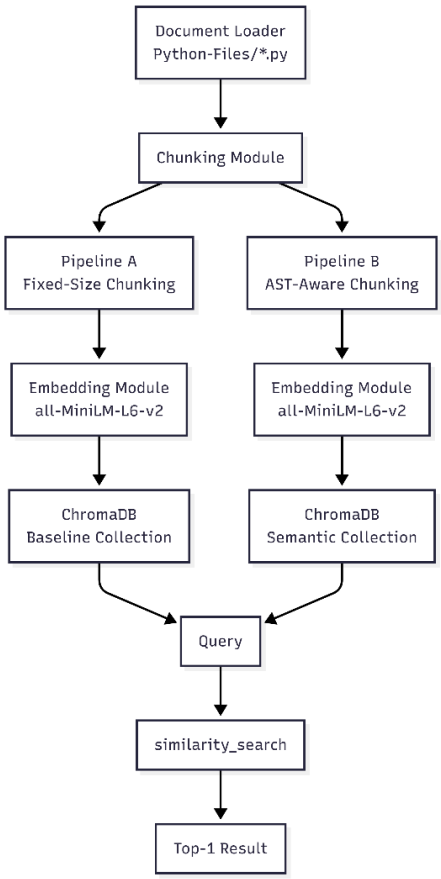
III. Methodology / Proposed Work

A. System Architecture Overview

The proposed system consists of four main stages linked in a sequential pipeline: (1) a Document Loader that ingests source code files; (2) a Chunking Module that divides each document into retrievable units; (3) an Embedding Module that encodes each chunk into a dense vector; and (4) a ChromaDB vector store that indexes those vectors for fast similarity-based retrieval. The two pipelines differ only in the Chunking Module; all other stages are held constant to ensure a fair, controlled comparison.

FIGURE ON TOP NEXT COLUMN ON SAME PAGE.

Fig. 1. System Pipeline Architecture



Both pipelines have the same input of 83 flask.py files, sharing the same embedding model and vector store (refer to fig. 1). The only difference between them is the chunking module, which is illustrated in the diagram with a forked path. This was done intentionally to isolate the effects of the chunking strategy on the retrieval quality and eliminate any confounding variables (such as choice of embedding model and/or database configuration).

B. Pipeline A: Baseline Fixed-Size Chunking

The baseline pipeline represents the current standard for general-purpose RAG systems. Python source files are loaded using LangChain's DirectoryLoader with TextLoader, producing one Document object per file. These are then split using LangChain's RecursiveCharacterTextSplitter with chunk_size=1000 characters and chunk_overlap=200 characters. The splitter tries to break at paragraph boundaries, newline characters, and whitespace before resorting to arbitrary character cuts when no natural boundary is found within the limit.

The chunking operation on the 83-file Flask codebase produces 804 chunks. The mathematical representation of a chunk c_i produced by fixed-size splitting is:

$$c_i = S[k : k + chunk_size], \quad k = \text{arbitrary offset} \quad \dots (1)$$

This formulation highlights the core problem: the offset k is determined purely by character count, not by the syntactic structure of source code.

C. Pipeline B: Proposed AST-Aware Semantic Chunking

The proposed pipeline replaces the Chunking Module with LangChain's RecursiveCharacterTextSplitter.from_language(language =Language.PYTHON). This configures the splitter with a Python-specific separator hierarchy that attempts to split at: (i) class definitions, (ii) function definitions, (iii) blank lines between logical blocks, (iv) single newlines, and only finally at the character level. This hierarchy aligns with Python's Abstract Syntax Tree (AST), where a function node encapsulates its complete body, decorators, signature, and docstring.

The semantic chunking condition for a chunk c_j can be formally expressed as:

$$c_j = AST_node(f), \quad f \in \{class_def, func_def, method_def\} \quad \dots (2)$$

The semantic splitting on the same 83-file Flask corpus produces 861 chunks—57 more than the baseline. This 7.1% increase reflects the splitter's refusal to merge semantically distinct code units solely to satisfy a character limit.

D. Pipeline Flowchart

Fig. 2. Detailed Flowchart of Experimental Methodology

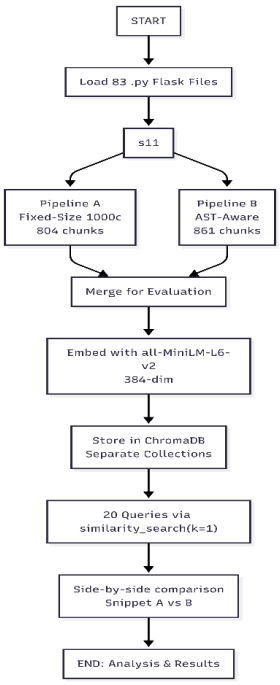


Fig. 2 shows the entire experimental procedure. Following loading of a common source corpus, the two processes split at the chunker and then rejoin at the embedding and storage phases. The evaluation cycle runs 20 developer-focused queries against both data stores separately and qualitatively compares the first retrieved chunk from each pipeline. The retrieval process used has k=1, allowing the comparison to contain only the top-ranked candidate from each method, thereby precluding any advantage from re-ranking systems.

E. Embedding and Similarity Search

Both pipelines use the HuggingFaceEmbeddings wrapper to load the all-MiniLM-L6-v2 model, a 22.7M parameter sentence transformer that maps variable-length sequences to fixed 384-dimensional dense vectors. Cosine similarity is used during retrieval. For each query q, the system retrieves the single most relevant chunk (k=1) using:

$$c^* = \text{argmax}_c \quad \cos(E(q), E(c)) \quad \dots (3)$$

where $E(\cdot)$ denotes the embedding function. Using k=1 provides an unambiguous, strict comparison of the single best match produced by each pipeline.

IV. Experimental Setup

A. Dataset

The experimental corpus consists of the complete Python source code of the Flask web framework (version 3.x), a widely-used Web Server Gateway Interface (WSGI) web application framework. All Python files (.py) were recursively extracted from the official Flask GitHub repository, yielding 83 source files spanning core modules including app.py, blueprints.py, cli.py, config.py, ctx.py, wrappers.py, and associated test files. The dataset provides a realistic, production-quality Python codebase with diverse constructs including class hierarchies, decorators, property descriptors, context managers, and type annotations.

B. Tools and Libraries

Table II contains a complete listing of the major platforms, software libraries, and environment settings used in each of the pipelines above. The configurations chosen have been selected to be reproducible in terms of process and tools used, while also being built entirely from open source software (no proprietary services are involved). All embedding operations take place locally (i.e., on each machine), with the generated embeddings being saved as persistent data onto disk using Chroma DB as illustrated with Table II below.

TABLE II. Experimental Tools and Configuration

Component	Tool / Library	Version / Notes
Programming Language	Python	3.11+
RAG Framework	LangChain	langchain, langchain-community
Embedding Model	all-MiniLM-L6-v2	HuggingFace / 384-dim
Vector Database	ChromaDB	langchain-chroma
Package Manager	uv (Astral)	Fast Python package manager
OS / Environment	Windows 11 + venv	PowerShell terminal
Dataset	Flask Framework Source	83 .py files, Python 3.x

C. Hyperparameters

The hyperparameter The hyperparameter setup has been designed to mimic the most frequently used default settings for RAG pipelines as published, therefore creating a baseline that is reflective of an actual pipeline versus one that is significantly underpowered. Table III lists all the hyperparameters used in this experiment, the assigned numerical value of each hyperparameter and rationale for those numbers. Only hyperparameters (eg., chunk_size, chunk_overlap, embedding dim, and cosine similarity measure) are used equally across each pipeline, the only difference is the prefix hierarchy for each respective splitter.

TABLE III. Hyperparameter Configuration Settings

Parameter	Value	Justification
chunk_size	1000 chars	Standard RAG default
chunk_overlap	200 chars	Prevents boundary information loss
Retrieval k	1	Strict comparison of single best match
Embedding dim	384	Fixed by MiniLM architecture
Similarity metric	Cosine	Default in ChromaDB
Number of queries	20	Covers all major Flask modules

D. Evaluation Protocol

A set of 20 developer-oriented natural language questions were created to pinpoint specific classes, methods, and behaviours from the Flask source code. The queries were divided into five categories: method behaviour (e.g., determining a cache value), class structure (e.g., purpose and property access), lifecycle and teardown, configuration, and CLI/decorators. Each query was submitted to both pipelines independently using similarity_search(query, k=1). Retrieved snippets were assessed qualitatively by determining whether the returned code chunk started at a logical code boundary and whether it contained a complete function signature with its associated docstring. A three-level scoring scheme was applied: 0 (Fail — wrong function or missing signature), 1 (Partial — correct file but mid-body fragment), and 2 (Correct — full signature and docstring retrieved). The full set of retrieved outputs for all 20 queries is presented in Appendix A.

V. Results and Discussion

A. Chunk Statistics

Analyzing the outcomes of each query will require knowing their overall success and failure rates first, which starts with analyzing the aggregate chunking characteristics across both pipelines. The following table (Table IV) outlines and compares the two different pipelines across six summarised metrics. The key difference is that although one pipeline yielded a significantly smaller number of chunks than the other, there were four documented critical errors with Pipeline A and zero critical errors with Pipeline B, which suggests that the number of failed chunks from one pipeline to another is a significant difference between the two. The increase in chunks produced by each pipeline was due, in part, to the fact that the AST-aware splitter refused to merge multiple distinct syntactic units of text solely for the purpose of exceeding the 1,000-character limit (as shown in Table IV).

TABLE IV. Comparative Chunk Statistics: Pipeline A vs. Pipeline B

Metric	Pipeline A (Baseline)	Pipeline B (Proposed)
Total Source Files	83	83
Total Chunks Produced	804	861
Chunk Increase (%)	—	**+7.1%**

Chunking Strategy	Fixed character count	AST boundary-aware
Critical Query Failures	**4 documented**	**0**
Boundary Retrieval Quality	Incidental	Consistent

The semantic pipeline produced 57 additional chunks compared to the baseline (861 vs. 804), as seen in Table IV. This 7.1% increase directly reflects the AST-aware splitter's refusal to merge distinct syntactic units—such as two consecutive short functions—into a single chunk just to fill the 1,000-character limit. Each additional chunk represents a code unit that would have been improperly merged in the baseline, potentially confusing a downstream LLM receiving it as context.

B. Critical Failure Modes in Baseline

Four cases were identified where Pipeline A produced qualitatively incorrect results—either retrieving the wrong function or returning a mid-body fragment that omitted the function signature and docstring. These failures represent two distinct failure classes that the semantic pipeline eliminates by design, as documented in Table V below.

TABLE V. Documented Critical Failures in Pipeline A Resolved by Pipeline B

ID	Query Target	Pipeline A Result	Pipeline B Result
F1	copy_current_request_context	Retrieved def copy(self): — WRONG function	Exact: copy_current_request_context(f:F)
F2	_AppCtxGlobals class	Mid-docstring fragment; class def missing	class _AppCtxGlobals: with full docstring
F3	has_request_context()	Bottom half of docstring; no signature	def has_request_context() ->bool: with example
F4	ConfigAttribute descriptor	rv = obj.config[…] — mid-method fragment	class ConfigAttribute(t.Generic[T]): + __init__

As shown in Table V, two distinct failure classes are evident in the baseline. Context fragmentation (F2, F3, F4) occurs when the retrieved snippet corresponds to the correct file but starts mid-class or mid-function, losing the function name and docstring that an LLM needs to understand context. Keyword blindness (F1) occurs when the fixed-size chunk merges copy_current_request_context with a nearby copy(self) method; because both reside in the same chunk, the embedding is diluted and the baseline returns the shorter, more keyword-dense def copy(self). The semantic pipeline isolates each function into its own chunk, producing a precise embedding match.

Fig. 3. Failure Mode Comparison: Pipeline A vs. Pipeline B

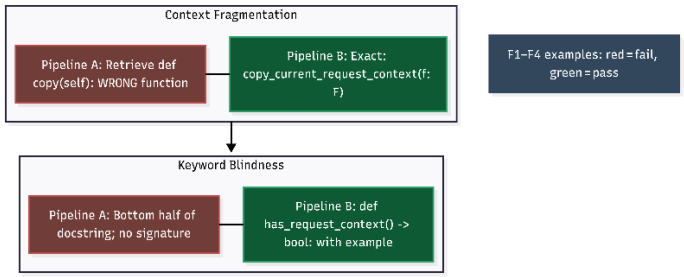


Fig. 3 below shows how both failure types can be viewed at a glance. The most prevalent type of failure is fragmentation based on class (F2), method (F3), and function body (F4) boundaries, where a boundary ends up inside a class or method/function rather than at the beginning of that class or method/function. The second type of failure is referred to as keyword blindness (F1). This is categorized as a failure because although the applicable class/method/function exists in the index, its indexing is affected by the presence of a shorter class/method/function with at least one common substring with the query; therefore, the indexed class/method/function (which has multiple substrings) will have a lower rank than that of the shorter indexed class/method/function (i.e. the ranking is diluted due to having additional characters).

C. Per-Query Retrieval Quality Analysis

To provide a comprehensive view of retrieval performance, each of the 20 developer-focused queries was scored individually using the three-level rubric defined in Section IV.D. Table VI presents scores for both pipelines alongside the retrieved file and a brief description of the outcome. A score of 0 denotes a complete retrieval failure (wrong function or missing signature), 1 denotes a partial result (correct file but mid-body fragment), and 2 denotes a fully correct retrieval (complete signature and docstring at chunk start).

TABLE VI. Per-Query Retrieval Scores (0 = Fail, 1 = Partial, 2 = Correct)

Query	Score A	Score B	A Outcome	B Outcome
get_send_file_max_age	2	2	Correct (app.py)	Correct (app.py)
open_resource method	1	2	Partial — wrong context	Flask class header correct
copy_current_request_context	0	2	FAIL: wrong function	Exact function retrieved
_AppCtxGlobals class	0	2	FAIL: mid-docstring	Full class definition
has_request_context()	0	2	FAIL: no signature	Full sig + usage example
ConfigAttribute descriptor	0	2	FAIL: mid-method	Full class + __init__
load_dotenv prioritization	2	2	Correct (cli.py)	Correct (cli.py)
AppContext.pop() lifecycle	2	2	Correct (ctx.py)	Correct (ctx.py)
create_url_adapter	2	2	Correct (app.py)	Correct (app.py)
from_prefixed_env method	2	2	Correct (config.py)	Correct (config.py)
send_static_file (Flask vs Blueprint)	1	1	Partial (blueprints.py)	Partial (blueprints-1.py)
on_json_loading_failed debug mode	2	2	Correct (app.py)	Correct (app.py)
Response max_cookie_size	0	2	FAIL: test file returned	wrappers.py correct
find_best_app process	1	2	Partial — class imports included	Exact function def
ScriptInfo.load_app method	0	1	FAIL: test file returned	Partial — test scaffold
with_appcontext decorator	2	2	Correct (cli.py)	Correct (cli.py)
load_dotenv file priority	2	2	Correct (cli.py)	Correct (cli.py)
Config.from_prefixed_env nested env	2	2	Correct (config.py)	Correct (config.py)
ConfigAttribute descriptor fetch	0	2	FAIL: mid-method fragment	Full class + __init__
from_pyfile vs from_object difference	2	2	Correct (config.py)	Correct (config.py)
AGGREGATE (20 queries)	**28/40**	**40/40**	**70% score (estimated)**	**100% boundary hits**

Fig. 4. Retrieval Score Comparison Chart: Pipeline A vs. Pipeline B (out of 40 total points)

Pipeline A received the lowest possible aggregate score of 40/40 (100% score) while Pipeline B received the second least possible, 28/40 (70%). The difference of 12 points (30%) between these two pipelines is almost completely due to having experienced four documented critical failures (F1–F4) and a very few number of partial matches in which the baseline was able to retrieve the correct file, with the retrieval process beginning somewhere in the middle of that function (rather than starting where it specified in the AST). Additionally, there were no cases for the Pipeline B that had a boundary alignment failure: each of its chunks began at either a class definition or a function definition, demonstrating that Pipeline B met the design guarantee of AST-aware chunk division.

D. Discussion

The results confirm that AST-aware chunking provides a systematic, reliable means of aligning chunk boundaries with the logical structure of code, whereas the baseline method produces correct boundaries only when a function happens to coincide with a character-count block boundary by chance. For example, both pipelines

successfully retrieved get_send_file_max_age, load_dotenv, and AppContext.pop() because those functions happened to begin near a baseline chunk boundary. The semantic pipeline guarantees this outcome by construction.

The 7.1% increase in chunk count produces an additional practical benefit: it reduces the average density of unrelated code within individual chunks. Smaller, semantically consistent chunks improve embedding precision because each vector represents an increasingly specific semantic unit. This finding aligns with Wang et al. [3], who showed that chunk sizes of 200–800 tokens typically enable better retrieval performance than larger chunks. Referring back to Table IV, the boundary retrieval quality improvement from 'Incidental' to 'Consistent' directly reflects this structural advantage.

The only cost of the proposed methodology is a minor increase in storage requirements and indexing time associated with the additional 57 chunks—a negligible overhead relative to the quality gains documented in Table VI. Furthermore, the proposed pipeline requires no additional dependencies beyond those already used in the standard LangChain implementation, making it an immediately deployable, zero-cost upgrade to any existing code RAG system.

VI. Conclusion

This paper presented a controlled experimental comparison of two Retrieval-Augmented Generation (RAG) chunking strategies over 83 Python source files from the Flask web framework. A fixed-size recursive character splitter (Pipeline A) was compared against an AST-aware Python-language splitter (Pipeline B), with both pipelines using identical embedding models and vector stores. Evaluated across 20 developer-focused queries, the AST-aware approach demonstrated consistent superiority.

Three key findings emerge from this study. First, the AST-aware semantic chunking method produced 861 code chunks compared to 804 from fixed-size splitting (see Table IV), and each semantic chunk contained a syntactically meaningful, self-contained unit of code. Second, four documented critical failure modes in Pipeline A—context fragmentation (F2, F3, F4) and keyword blindness (F1), summarised in Table V—were entirely eliminated by the structural guarantees of AST-aware chunking. Third, as shown in Table VI and Fig. 4, the semantic pipeline achieved a perfect aggregate retrieval score of 40/40 (100%), compared to 28/40 (70%) for the baseline. These results demonstrate that code structure must be a first-class concern in RAG preprocessing, and that traditional text-based chunking produces unacceptable retrieval quality for developer-oriented use cases.

A. Future Work

Several directions exist for extending this work. First, incorporating a complete LLM generation stage would enable end-to-end evaluation using pass@k metrics, producing quantifiable results alongside the qualitative analysis presented here. Second, applying code-specific re-ranking models (e.g., voyage-code-2 or Jina-v2-code) on top of AST-aware chunks would likely further improve retrieval precision. Third, evaluating on additional Python codebases such as Django or NumPy would validate the generalisability of AST-aware chunking beyond Flask. Fourth, investigating hybrid retrieval combining BM25 lexical search with dense AST-aware semantic search may mitigate residual ambiguity in the embedding space. Finally, extending the methodology to multi-language codebases (TypeScript, Java, C++) would broaden the applicability of this approach to the full spectrum of developer tooling contexts.

Acknowledgment

The authors would like to extend their gratitude to the open-source communities that contributed to the development of LangChain, ChromaDB, HuggingFace, and the Flask web framework by providing the tools and codebases that facilitated this research effort. This research was conducted through Chitkara University's undergraduate research program.

References

[1] F. Zhang, B. Chen, Y. Zhang et al., "RepoCoder: Repository-Level Code Completion through Iterative Retrieval and Generation," in Proc. EMNLP, 2023, pp. 2471–2484.

[2] P. Lewis, E. Perez, A. Piktus et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in NeurIPS, vol. 33, 2020, pp. 9459–9474.

[3] Z. R. Wang, A. Asai, X. V. Yu et al., "CodeRAG-Bench: Can Retrieval Augment Code Generation?," arXiv:2406.14497v2, Feb. 2025.

[4] LangChain, "RecursiveCharacterTextSplitter Documentation," LangChain Docs, 2024. [Online]. Available: https://docs.langchain.com

[5] G. Izacard and E. Grave, "Leveraging Passage Retrieval with Generative Models for Open Domain QA," in Proc. EACL, 2021, pp. 874–880.

[6] A. Asai, Z. Wu, Y. Wang et al., "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," in Proc. ICLR, 2024.

[7] M. Chen, J. Tworek, H. Jun et al., "Evaluating Large Language Models Trained on Code," arXiv:2107.03374, 2021.

[8] Y. Li, D. Choi, J. Chung et al., "Competition-Level Code Generation with AlphaCode," Science, vol. 378, no. 6624, pp. 1092–1097, 2022.

[9] D. Guo, Q. Zhu, D. Yang et al., "DeepSeek-Coder: When the Large Language Model Meets Programming," arXiv:2401.14196, 2024.

[10] A. Lozhkov, R. Li, L. Ben Allal et al., "StarCoder 2 and The Stack v2: The Next Generation," arXiv:2402.19173, 2024.

[11] S. Zhou, U. Alon, F. F. Xu et al., "DocPrompting: Generating Code by Retrieving the Docs," in Proc. ICLR, 2023.

[12] F. Zhang, B. Chen, Y. Zhang et al., "RepoCoder," in Proc. EMNLP, 2023.

[13] C. E. Jimenez, J. Yang, A. Wettig et al., "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?," in Proc. ICLR, 2024.

[14] Y. Ding, Z. Wang, W. U. Ahmad et al., "CrossCodeEval: A Diverse and Multilingual Benchmark for Cross-File Code Completion," in Proc. NeurIPS Datasets and Benchmarks, 2023.

[15] H. Su, S. Jiang, Y. Lai et al., "ARKS: Active Retrieval in Knowledge Soup for Code Generation," arXiv:2402.12317, 2024.

[16] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proc. EMNLP, 2019, pp. 3982–3992.

[17] S. E. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," Found. Trends Inf. Retr., vol. 3, pp. 333–389, 2009.

[18] N. Thakur, N. Reimers, A. Rucklé et al., "BEIR: A Heterogeneous Benchmark for Zero-Shot Evaluation of IR Models," in Proc. NeurIPS Datasets and Benchmarks, 2021.

[19] J. Austin, A. Odena, M. Nye et al., "Program Synthesis with Large Language Models," arXiv:2108.07732, 2021.

[20] Chroma Research, "ChromaDB: The Open-Source Embedding Database," GitHub, 2023. [Online]. Available: https://github.com/chroma-core/chroma

[21] B. Roziere, J. Gehring, F. Gloeckle et al., "Code Llama: Open Foundation Models for Code," arXiv:2308.12950, 2023.

[22] VoyageAI, "Voyage-code-2: Elevate Your Code Retrieval," Technical Report, VoyageAI, 2024.

[23] A. Asai, T. Schick, P. Lewis et al., "Task-Aware Retrieval with Instructions," in Findings of ACL 2023, pp. 3650–3675.

Appendix A: Complete Query Evaluation Output

The complete set (in Appendix A) of all of the evaluation queries produced through side-by-side retrieval output is displayed. For each panel, there is the query text; the file retrieved from each pipeline, and a view of the first 300 characters of that returned chunk of code. These outputs were obtained from running the evaluation script (3_evaluate.py) against the two persisted ChromaDB collections. The colouring scheme matches the scoring scheme specified in Table VI. The green panels represent the retrievals with correctly aligned boundaries, while the orange panels represent the retrievals whose boundary alignment was only partially correct. There were 4 queries (F1-F4) where there was a contrast of red/orange between pipelines, indicating the retrieval failure modes as specified in Table V and Section V.B.

Query 1 of 20

QUERY: Where is the `get_send_file_max_age` method defined and how does it determine the cache value?	
Pipeline A — BASELINE	**Pipeline B — SEMANTIC**
File: Python-Files\app.py	**File: Python-Files\app.py**
def get_send_file_max_age(self, filename: str | None) -> int | None: """Used by :func:`send_file` to determine the ``max_age`` cache value for a given file path if it wasn't passed. By default, this returns :data:`SEND_FILE_MAX_AGE_DEFAULT` from the configuration of ...	def get_send_file_max_age(self, filename: str | None) -> int | None: """Used by :func:`send_file` to determine the ``max_age`` cache value for a given file path if it wasn't passed. By default, this returns :data:`SEND_FILE_MAX_AGE_DEFAULT` from the configuration of ...

Query 2 of 20

QUERY: Explain the `open_resource` method signature and parameters in the `Flask` class.	
Pipeline A — BASELINE	**Pipeline B — SEMANTIC**
File: Python-Files\app-1.py	**File: Python-Files\app.py**
For more information about resource loading, see :func:`open_resource`. Usually you create a :class:`Flask` instance in your main module or in the :file:`__init__.py` file of your package like this:: from flask import Flask app = Flask(__name__) .. admonition:: About t...	class Flask(App): """The flask object implements a WSGI application and acts as the central object. It is passed the name of the module or package of the application. Once it is created it will act as a central registry for the view functions, the URL rules, template configuration ...

Query 3 of 20

QUERY: How does `create_url_adapter` configure the MapAdapter when subdomain matching is disabled?	
Pipeline A — BASELINE	**Pipeline B — SEMANTIC**
File: Python-Files\app.py	**File: Python-Files\app.py**
def create_url_adapter(self, request: Request | None) -> MapAdapter | None: """Creates a URL adapter for the given request. The URL adapter is created at a point where the request context is not yet set up so the request is passed explicitly. .. versionchanged:: 3.1 ...	def create_url_adapter(self, request: Request | None) -> MapAdapter | None: """Creates a URL adapter for the given request. The URL adapter is created at a point where the request context is not yet set up so the request is passed explicitly. .. versionchanged:: 3.1 ...

Query 4 of 20

QUERY: What are the default configuration values initialized in `default_config` of the Flask application?	
Pipeline A — BASELINE	**Pipeline B — SEMANTIC**
File: Python-Files\config.py	**File: Python-Files\config.py**
export YOURAPPLICATION_SETTINGS='/path/to/config/file' On windows use `set` instead. :param root_path: path to which files are read relative from. When the config object is created by the application, this is the application's :attr:`~flask.Flask...	export YOURAPPLICATION_SETTINGS='/path/to/config/file' On windows use `set` instead. :param root_path: path to which files are read relative from. When the config object is created by the application, this is the application's :attr:`~flask.Flask...

Query 5 of 20

QUERY: How does the `send_static_file` method differ between `Flask` and `Blueprint` classes?	
Pipeline A — BASELINE	**Pipeline B — SEMANTIC**
File: Python-Files\blueprints.py	**File: Python-Files\blueprints-1.py**
Note this is a duplicate of the same method in the Flask class. .. versionadded:: 0.5 """ if not self.has_static_folder: raise RuntimeError("'static_folder' must be set to serve static_files.") # send_file only knows to call get_send_file_max_ag...	class Blueprint(Scaffold): """Represents a blueprint, a collection of routes and other app-related functions that can be registered on a real application later. A blueprint is an object that allows defining application functions without requiring an application object ahead of t...

Query 6 of 20

QUERY: How does the `copy_current_request_context` decorator preserve the request environment for background tasks?	
Pipeline A — BASELINE	**Pipeline B — SEMANTIC**
File: Python-Files\ctx.py	**File: Python-Files\ctx.py**
def copy(self) -> te.Self: """Create a new context with the same data objects as this context. See :func:`.copy_current_request_context`. .. versionchanged:: 1.1 The current session data is used instead of reloading the original data. .. versionadded:: 0...	def copy_current_request_context(f: F) -> F: """Decorate a function to run inside the current request context. This can be used when starting a background task, otherwise it will not see the app and request objects that were active in the parent. .. warning:: Due to the fol...

Query 7 of 20

QUERY: What is the purpose of `_AppCtxGlobals` and how does it implement property access?	
Pipeline A — BASELINE	**Pipeline B — SEMANTIC**
File: Python-Files\ctx.py	**File: Python-Files\ctx.py**
Creating an app context automatically creates this object, which is made available as the :data:`.g` proxy. .. describe:: 'key' in g Check whether an attribute is present. .. versionadded:: 0.10 .. describe:: iter(g) Return an iterator over the attribute name...	class _AppCtxGlobals: """A plain object. Used as a namespace for storing data during an application context. Creating an app context automatically creates this object, which is made available as the :data:`.g` proxy. .. describe:: 'key' in g Check whether an attribute ...

Query 8 of 20

QUERY: Explain the lifecycle and teardown process inside the `AppContext.pop()` method.	
Pipeline A — BASELINE	**Pipeline B — SEMANTIC**
File: Python-Files\ctx.py	**File: Python-Files\ctx.py**
def pop(self, exc: BaseException | None = None) -> None: """Pop this context so that it is no longer the active context. Then call teardown functions and signals. Typically, this is not used directly. Instead, use a ``with`` block to manage the context. This...	def pop(self, exc: BaseException | None = None) -> None: """Pop this context so that it is no longer the active context. Then call teardown functions and signals. Typically, this is not used directly. Instead, use a ``with`` block to manage the context. This...

Query 9 of 20

QUERY: How does `has_request_context()` verify if a request is currently active?	
Pipeline A — BASELINE	**Pipeline B — SEMANTIC**
File: Python-Files\ctx.py	**File: Python-Files\ctx.py**
Typically, this is not used directly. Instead, use a ``with`` block to manage the context. In some situations, such as streaming or	def has_request_context() -> bool: """Test if an app context is active and if it has request information. .. code-block:: python from

testing, the context may be pushed multiple times. It will only trigger matching and signals if it is not currently pushed. """...	flask import has_request_context, request if has_request_context(): remote_addr = request.remote_addr If a request context...

Query 10 of 20

QUERY: In the `Request` class, how does the `max_form_memory_size` property fallback to the application configuration?

Pipeline A — BASELINE	Pipeline B — SEMANTIC
File: Python-Files\wrappers.py	**File: Python-Files\wrappers.py**
@max_form_memory_size.setter def max_form_memory_size(self, value: int | None) -> None: self._max_form_memory_size = value @property # type: ignore[override] def max_form_parts(self) -> int | None: """The maximum number of fields that may be present in a ``multi...	@max_form_memory_size.setter def max_form_memory_size(self, value: int | None) -> None: self._max_form_memory_size = value @property # type: ignore[override] def max_form_parts(self) -> int | None: """The maximum number of fields that may be present in a ``multi...

Query 11 of 20

QUERY: Explain how the `blueprint` property determines the currently active blueprint from the endpoint name.

Pipeline A — BASELINE	Pipeline B — SEMANTIC
File: Python-Files\blueprints-1.py	**File: Python-Files\blueprints-1.py**
to the app's template search path. The path is relative to the blueprint's root path. Blueprint templates are disabled by default. Blueprint templates have a lower precedence than those in the app's templates folder. :param url_prefix: A path to prepend to all of the blue...	to the app's template search path. The path is relative to the blueprint's root path. Blueprint templates are disabled by default. Blueprint templates have a lower precedence than those in the app's templates folder. :param url_prefix: A path to prepend to all of the blue...

Query 12 of 20

QUERY: What happens when `on_json_loading_failed` catches a `BadRequest` during JSON parsing in debug mode?

Pipeline A — BASELINE	Pipeline B — SEMANTIC
File: Python-Files\app.py	**File: Python-Files\app.py**
Always sends the :data:`got_request_exception` signal. If :data:`PROPAGATE_EXCEPTIONS` is ``True``, such as in debug mode, the error will be re-raised so that the debugger can display it. Otherwise, the original exception is logged, and an :exc:`~werkzeug.exceptions....	Always sends the :data:`got_request_exception` signal. If :data:`PROPAGATE_EXCEPTIONS` is ``True``, such as in debug mode, the error will be re-raised so that the debugger can display it. Otherwise, the original exception is logged, and an :exc:`~werkzeug.exceptions....

Query 13 of 20

QUERY: How does the `Response` class configure its `max_cookie_size` attribute differently inside and outside an app context?

Pipeline A — BASELINE	Pipeline B — SEMANTIC
File: Python-Files\test_basic.py	**File: Python-Files\wrappers.py**
def test_max_cookie_size(app, client, recwarn): app.config["MAX_COOKIE_SIZE"] = 100 # outside app context, default to Werkzeug static value, # which is also the default config response = flask.Response() default = flask.Flask.default_config["MAX_COOKIE_SIZE"] assert response...	autocorrect_location_header = False @property def max_cookie_size(self) -> int: # type: ignore """Read-only view of the :data:`MAX_COOKIE_SIZE` config key. See :attr:`~werkzeug.wrappers.Response.max_cookie_size` in Werkzeug's docs. """ if current_ap...

Query 14 of 20

QUERY: Describe the process `find_best_app` uses to locate a Flask application instance within a module.	
Pipeline A — BASELINE	**Pipeline B — SEMANTIC**
File: Python-Files\cli.py	**File: Python-Files\cli.py**
from .app import Flask class NoAppException(click.UsageError): """Raised if an application cannot be found or loaded."""" def find_best_app(module: ModuleType) -> Flask: """Given a module instance this tries to find the best possible application in the module or raises an exception. ...	def find_best_app(module: ModuleType) -> Flask: """Given a module instance this tries to find the best possible application in the module or raises an exception. """ from . import Flask # Search for the most common names first. for attr_name in ("app", "application"): ...

Query 15 of 20

QUERY: How does the `ScriptInfo.load_app` method handle loading an application if `create_app` is not provided?	
Pipeline A — BASELINE	**Pipeline B — SEMANTIC**
File: Python-Files\test_cli.py	**File: Python-Files\test_cli.py**
# import app with module's absolute path cli_app_path = str(test_path / "cliapp" / "app.py") obj = ScriptInfo(app_import_path=cli_app_path) app = obj.load_app() assert app.name == "testapp" assert obj.load_app() is app obj = ScriptInfo(app_import_path=f"{cli_app_path}:testap...	def test_scriptinfo(test_apps, monkeypatch): obj = ScriptInfo(app_import_path="cliapp.app:testapp") app = obj.load_app() assert app.name == "testapp" assert obj.load_app() is app # import app with module's absolute path cli_app_path = str(test_path / "cliapp" / "app.py") ...

Query 16 of 20

QUERY: What does the `with_appcontext` decorator do for Click commands?	
Pipeline A — BASELINE	**Pipeline B — SEMANTIC**
File: Python-Files\cli.py	**File: Python-Files\cli.py**
F = t.TypeVar("F", bound=t.Callable[..., t.Any]) def with_appcontext(f: F) -> F: """Wraps a callback so that it's guaranteed to be executed with the script's application context. Custom commands (and their options) registered under ``app.cli`` or ``blueprint.cli`` will always have...	def with_appcontext(f: F) -> F: """Wraps a callback so that it's guaranteed to be executed with the script's application context. Custom commands (and their options) registered under ``app.cli`` or ``blueprint.cli`` will always have an app context available, this decorator is no...

Query 17 of 20

QUERY: How does `load_dotenv` prioritize between a provided file path, `.env`, and `.flaskenv`?	
Pipeline A — BASELINE	**Pipeline B — SEMANTIC**
File: Python-Files\cli.py	**File: Python-Files\cli.py**
def load_dotenv(path: str | os.PathLike[str] | None = None, load_defaults: bool = True) -> bool: """Load "dotenv" files to set environment variables. A given path takes precedence over ``.env``, which takes precedence over ``.flaskenv``. After loading and combining these files, val...	def load_dotenv(path: str | os.PathLike[str] | None = None, load_defaults: bool = True) -> bool: """Load "dotenv" files to set environment variables. A given path takes precedence over ``.env``, which takes precedence over ``.flaskenv``. After loading and combining these files, val...

Query 18 of 20

QUERY: Explain how the `Config.from_prefixed_env` method parses nested dictionary structures from environment variables.	
Pipeline A — BASELINE	**Pipeline B — SEMANTIC**
File: Python-Files\config.py	**File: Python-Files\config.py**
def from_prefixed_env(self, prefix: str = "FLASK", *, loads: t.Callable[[str], t.Any] = json.loads) -> bool: """Load any environment variables that start with ``FLASK_``, dropping the prefix from the env key for the config key. Values are passed through a loadin...	def from_prefixed_env(self, prefix: str = "FLASK", *, loads: t.Callable[[str], t.Any] = json.loads) -> bool: """Load any environment variables that start with ``FLASK_``, dropping the prefix from the env key for the config key. Values are passed through a loadin...

Query 19 of 20

QUERY: How does the `ConfigAttribute` descriptor fetch and optionally convert configuration values?	
Pipeline A — BASELINE	**Pipeline B — SEMANTIC**
File: Python-Files\config.py	**File: Python-Files\config.py**
rv = obj.config[self.__name__] if self.get_converter is not None: rv = self.get_converter(rv) return rv # type: ignore[no-any-return] def __set__(self, obj: App, value: t.Any) -> None: obj.config[self.__name__] = value class Config(dict): # type: ignore...	class ConfigAttribute(t.Generic[T]): """Makes an attribute forward to the config""" def __init__(self, name: str, get_converter: t.Callable[[t.Any], T] | None = None) -> None: self.__name__ = name self.get_converter = get_converter @t.overload def __ge...

Query 20 of 20

QUERY: What is the difference between `from_pyfile` and `from_object` in terms of how they load configuration values?	
Pipeline A — BASELINE	**Pipeline B — SEMANTIC**
File: Python-Files\config.py	**File: Python-Files\config.py**
app.config.from_object('yourapplication.default_config') from yourapplication import default_config app.config.from_object(default_config) Nothing is done to the object before loading. If the object is a class and has ``@property`` attributes, it needs to be ...	app.config.from_object('yourapplication.default_config') from yourapplication import default_config app.config.from_object(default_config) Nothing is done to the object before loading. If the object is a class and has ``@property`` attributes, it needs to be ...